

# Operations Guide

---

## 1. Introduction

Dokumen ini berisi panduan operasional untuk melakukan monitoring, observability, troubleshooting, dan investigasi masalah pada sistem microservices Aplikasi Kelarin.

Panduan ini digunakan oleh tim Developer, QA, maupun DevOps untuk memastikan sistem tetap berjalan dengan baik serta mempermudah proses identifikasi masalah ketika terjadi gangguan.

---

## 2. Service Overview

---

| Service         | Port | Description                      |
|-----------------|------|----------------------------------|
| Frontend        | 80   | Antarmuka pengguna aplikasi      |
| Gateway (Nginx) | 80   | API Gateway dan reverse proxy    |
| Auth Service    | 8001 | Authentication dan authorization |
| Task Service    | 8002 | Task management service          |
| Auth Database   | 5432 | Database Auth Service            |
| Task Database   | 5432 | Database Task Service            |

---

## 3. Health Check Guide

---

### Check Seluruh Container

```
docker compose ps
```

Expected Result:

```
STATUS = Up (healthy)
```

---

### Check Auth Service Health

```
http://localhost:8001/health
```

Expected Response:

```
{  
  "status": "healthy"  
}
```

---

## Check Task Service Health

```
http://localhost:8002/health
```

Expected Response:

```
{  
  "status": "healthy"  
}
```

Apabila dependency mengalami gangguan:

```
{  
  "status": "degraded",  
  "dependencies": {  
    "auth-service": "unhealthy"  
  }  
}
```

---

## 4. Log Monitoring Guide

### Melihat Seluruh Log

```
docker compose logs
```

---

### Melihat Log Auth Service

```
docker compose logs auth-service
```

---

## Melihat Log Task Service

```
docker compose logs task-service
```

---

## Monitoring Log Real-Time

```
docker compose logs -f task-service
```

atau

```
docker compose logs -f auth-service
```

Keterangan:

- `-f` digunakan untuk mengikuti log secara real-time.
- Berguna saat melakukan debugging maupun reliability testing.

---

## 5. Correlation ID Tracing Guide

---

### Tujuan

Correlation ID digunakan untuk melacak satu request yang melewati beberapa service sehingga proses investigasi dan debugging menjadi lebih mudah.

---

### Contoh Log

```
{
  "timestamp": "2026-06-11T15:02:27",
  "level": "INFO",
  "message": "GET /health",
  "correlation_id": "f7930859-63aa-4029-ba45-965cf4bde3dc"
}
```

---

### Cara Trace Request

Cari Correlation ID pada log:

```
docker compose logs task-service
```

Contoh:

```
correlation_id=f7930859-63aa-4029-ba45-965cf4bde3dc
```

Gunakan ID tersebut untuk mencari seluruh aktivitas request yang sama pada service lain sehingga alur request dapat ditelusuri secara end-to-end.

---

## 6. Metrics Monitoring Guide

---

### Tujuan

Metrics digunakan untuk memantau performa aplikasi serta membantu mendeteksi masalah lebih awal.

---

### Check Metrics Endpoint

Auth Service:

```
http://localhost:8001/metrics
```

Task Service:

```
http://localhost:8002/metrics
```

Endpoint metrics akan menampilkan data monitoring yang dapat digunakan untuk observability dan analisis performa layanan.

---

### Available Metrics

Sistem menyediakan endpoint metrics untuk memantau performa layanan secara real-time.

| Metrics          | Description                          |
|------------------|--------------------------------------|
| Request Count    | Jumlah request yang diterima service |
| Request Duration | Waktu pemrosesan request             |
| Error Count      | Jumlah request yang gagal            |

| Metrics           | Description                   |
|-------------------|-------------------------------|
| Service Health    | Status kesehatan service      |
| Dependency Status | Status layanan yang terhubung |

## Logging Structure

Sistem menggunakan structured logging untuk mempermudah observability dan troubleshooting.

Contoh format log:

```
{
  "timestamp": "2026-06-11T15:02:27",
  "level": "INFO",
  "message": "GET /health",
  "correlation_id": "f7930859-63aa-4029-ba45-965cf4bde3dc"
}
```

Keterangan:

| Field          | Description                        |
|----------------|------------------------------------|
| timestamp      | Waktu request diproses             |
| level          | Tingkat log (INFO, WARNING, ERROR) |
| message        | Informasi aktivitas service        |
| correlation_id | ID unik untuk tracing request      |

## 7. Common Troubleshooting

### Service Tidak Berjalan

#### Gejala

Container tidak muncul pada hasil:

```
docker compose ps
```

#### Solusi

```
docker compose up --build
```

---

## Service Unhealthy

### Gejala

```
STATUS = unhealthy
```

### Solusi

Periksa log:

```
docker compose logs <service-name>
```

Pastikan database dan dependency service berjalan normal.

---

## Gateway Tidak Dapat Mengakses Service

### Gejala

```
502 Bad Gateway
```

atau

```
504 Gateway Timeout
```

### Solusi

Periksa:

```
docker compose ps
```

Pastikan service tujuan masih aktif dan dapat diakses.

---

## Database Connection Error

### Gejala

```
Connection refused
```

atau

```
Database unavailable
```

## Solusi

Pastikan container database aktif:

```
docker compose ps
```

Periksa konfigurasi environment variable dan network Docker.

---

## Health Status Degraded

### Gejala

```
{  
  "status": "degraded"  
}
```

## Solusi

Identifikasi dependency yang gagal kemudian cek log service terkait.

---

## 8. Escalation Path

| Level   | Role              | Responsibility                                                                               |
|---------|-------------------|----------------------------------------------------------------------------------------------|
| Level 1 | QA Engineer       | Melakukan reproduksi bug, mengumpulkan bukti error, dan mendokumentasikan hasil pengujian    |
| Level 2 | Backend Developer | Investigasi service error, analisis log aplikasi, dan perbaikan endpoint atau business logic |
| Level 3 | DevOps Engineer   | Investigasi container, deployment, resource server, dan proses recovery service              |
| Level 4 | Project Lead      | Pengambilan keputusan teknis, koordinasi tim, dan penentuan prioritas penanganan insiden     |

## 9. Operational Checklist

---

| Activity                | Frequency  |
|-------------------------|------------|
| Check Health Endpoint   | Daily      |
| Check Container Status  | Daily      |
| Review Application Logs | Daily      |
| Monitor Error Rate      | Daily      |
| Verify Metrics          | Weekly     |
| Reliability Testing     | Per Sprint |

---

## 10. Conclusion

---

Panduan operasional ini digunakan sebagai referensi utama dalam melakukan monitoring, observability, troubleshooting, dan investigasi masalah pada sistem microservices Kelarin.

Dengan adanya health monitoring, structured logging, correlation ID tracing, metrics monitoring, dan escalation path yang jelas, proses identifikasi serta penanganan masalah dapat dilakukan lebih cepat dan terstruktur.

---

## Final Status

---

● OPERATIONS GUIDE READY FOR USE